

9-RL Q-Learning

强化学习 RL

Q-Learning/主动强化学习

最终可以得到最优的**策略和价值**

需要权衡

- 探索Exploration 尝试未知的行动以获取信息
- 利用Exploitation 信息足够后，最终要运用已掌握的知识做出决策

Q-Value Iteration 基于模型的利用

类似价值迭代，从Q价值出发，需要已知转移模型T和奖励模型R

通常设初始化 $Q_0(s, a) = 0$

$$Q_{k+1}(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')]$$

直到 $Q_{k+1}(s, a)$ 稳定后，就是 $Q(s, a)$

公式的推导其实就是把迭代形式的贝尔曼方程反过来带入：

$$Q_{k+1}(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

其中

$$V_k(s') = \max_{a'} Q_k(s', a')$$

这是最核心的替换

Q-Learning 无模型学习

在探索中收集样本(s,a,s',r)，依旧通过**时序差分**进行更新

$$\text{sample} = R(s, a, s') + \gamma \max_{a'} Q(s', a')$$

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \cdot \text{sample}$$

这里的更新目标（max）是在利用已有知识，而数据采集过程包含探索

最终，**最优策略**由收敛后的Q值导出： $\pi^*(s) = \arg \max_a Q(s, a)$

特别的，如果知道是Q*，那么可以直接知道策略，因为qstate就是action后的价值（MDP笔记）[6-MDP 价值迭代](#)

另外，公式中的max保证了无论本次行动是什么，Q值都是根据目前为止最优的行动更新的，所以Q-Learning是不需要策略的 off-policy learning
只要探索的足够多，学习率一开始设置的足够小，且逐步减少（让更新量变小，保证收敛），这样子无论怎么选探索的行动都可以得到最优策略

强烈建议去看BV1HcqpYwEw6 最后20分钟的演示，讲解非常清楚

如何权衡探索和利用

Epsilon-Greedy

每一步都进行先进行一次判定，有 ϵ 的可能随机行动，剩下的可能按着（目前已经知道的）最优策略行动

ϵ 在一开始应该设置的比较大，从而多进行探索；一段时间后应当调小，才能反映出学习的成果（不然可能策略已经收敛，还在随机行动）

Exploration Function

显而易见更好的做法是，对于已经探索过足够次数的地方，就不在选择随机探索
用探索函数 $f(u,n) = u + \frac{k}{n}$ 代替 $\text{sample} = R(s,a,s') + \gamma \max_{a'} f(Q(s',a'), N(s',a'))$

其中u是价值，n是被探索的次数，k是一个常量

这个函数相当于给价值附带了一项奖励，当没怎么被探索时，奖励更大，更新的Q-Value就更大（准确的说这里的Q-Value多包含了一部分）

Approximate Q-Learning 近似Q学习

之前提到的都是精确的方法，进行Q-Learning的时候，要维护所有的Q-State，总数量是状态数×行动数，这可能完全无法容纳

近似方法的想法是，先学习一部分状态，遇到相似的新状态时根据已有经验来推断策略（泛化Generalize），而不是认为这是个完全不同的状态，这其实就是机器学习Machine Learning的思想

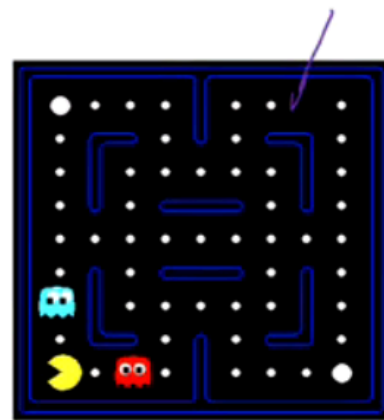
Let's say we discover through experience that this state is bad:



In naïve q-learning, we know nothing about this state:



Or even this one!



引入一个类似评估函数的方法，设置一系列的特征feature/property，对于吃豆人，可能是：

- Distance to closest ghost
- Distance to closest dot
- Number of ghosts
- $1/(\text{dist to dot})^2$
- ...

这些特征是把state映射到 $R \in [0, 1]$ 上的函数，然后对这些特征加权求和，作为Q-Value

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

这些权重通过类似Q-Learning的方法自动调整

对于一个样本 (s, a, s', r) ，预测的特征值 $Q(s, a)$ ，计算与样本给出的Q之间的差异

$$\text{fferee} = [r + \gamma \max_{a'} Q(s', a')] - Q(s, a)$$

然后遍历更新

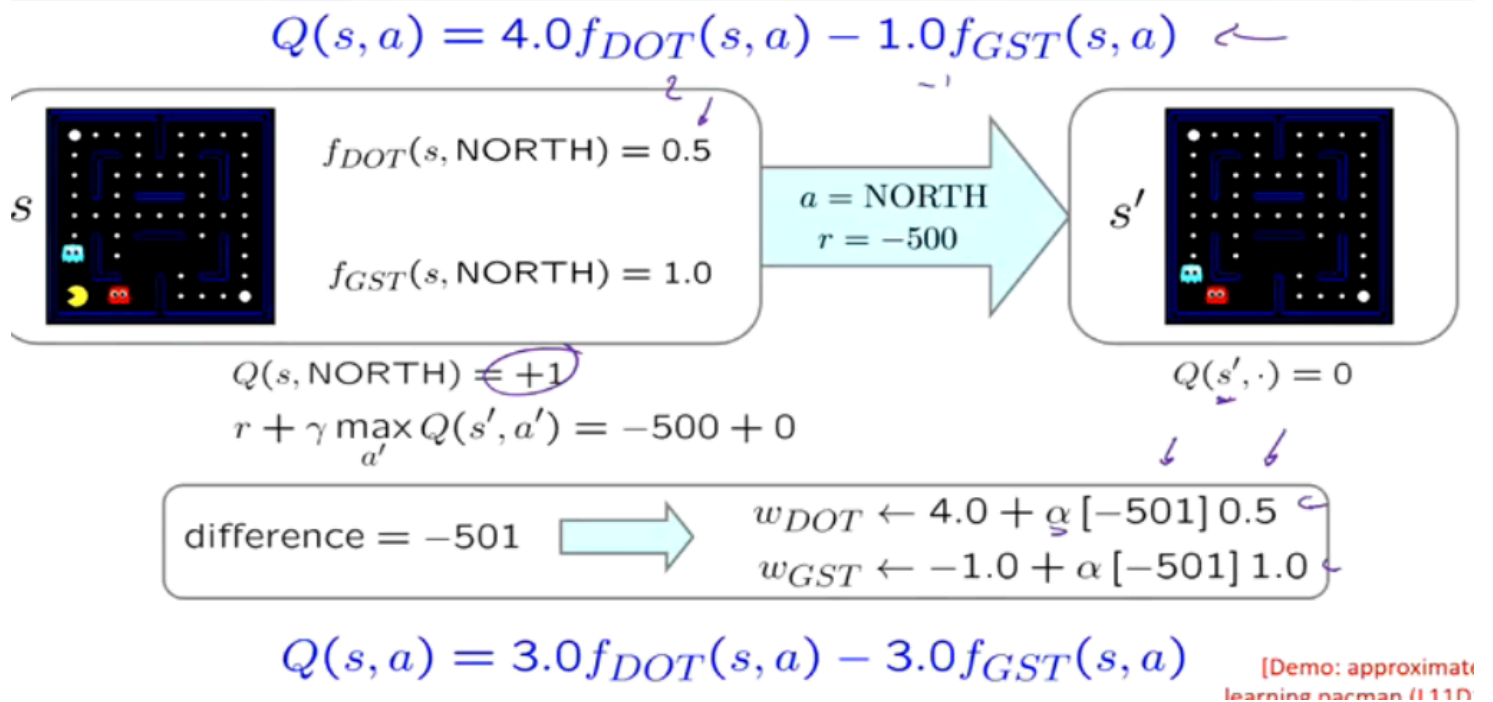
$$Q_i(s, a) \leftarrow Q_i(s, a) + \alpha \cdot \text{fferee}_i(s, a)$$

当 $\text{ff} < 0$ ，说明预测的Q更大，权重会被调小

$Q_i(s, a)$ 的大小影响改变量，这个样本下越活跃的特征，会主导权重的改变

这个公式的推导类似最小二乘法，此处略去。另外这个更新的方法就是一种机器学习

Example: Q-Pacman



这样就通过少量的特征权重数据概括了大量的Q-State，学习速度会快很多

但是如果特征太少或者不恰当，很可能让完全不相似的情况有一样的特征值，就没办法区分；如果特征太多，样本又不够多，可能会发生过拟合

Policy Search 策略搜索

之前针对Q价值进行学习，但其实数值本身不重要，重要的是Q价值的排序反映的action
比如吃豆人项目里，近似Q值可能非常离谱（比如在绝境给出+100），但只要它在危险动作上给的分数更低，在吃豆动作上给的分数更高，那给出的策略就是对的
为何不直接去学那个最大化reward的策略本身？这就是策略搜索

最简单的策略搜索，先从一个已经学习好的Q函数和策略出发，爬山法微调权重，再直接执行策略，如果效果更好就保留

但是分辨策略更好与否可能需要大量测试，在样本效率上比Q-Learning低，而且如果权重很多，这可能就不可能了

总结

已知的MDP：

- 求解V Q pi*：价值迭代/策略迭代
- 根据固定的pi评估价值：策略评估

未知的MDP：

- 基于模型：通过样本平均值求出T R，转换为已知情形
- 无模型：
 - 求解V Q pi*：Q-Learning（精确/近似）

- 根据固定的 π 评估价值：Value-Learning（直接评估/时间差分学习）
- 求解 π^* ：Policy Search